

The UML use case diagram describes the key actors and interaction scenarios of the eMagiz Ticketing System

Actors:

- System User - generic actor representing any authenticated user of the system. Specific roles such as Customer, Support Agent, and Internal Consultant specialize this actor and inherit basic use cases such as authentication and access to their tickets.
- Customer - external client who reports issues or submits requests. The Customer can browse the knowledge base, receive automated alerts, and initiate different types of tickets: RFCs and Incidents.
- Support Agent - support staff member responsible for assessing, processing, and managing tickets. The Support Agent can create internal tickets, log action details, accept and deny tickets, and assign tickets to handlers.
- Internal Consultant - can view organization-wide tickets, inspect the audit log, and manage tickets assigned to them.
- eMagiz Platform – external platform integrated with the ticketing system. The platform can initiate ticket creation, for example, via automated alerts or system integrations, and therefore participates in the “Submit Ticket” use case.

Main use cases

- Authenticate User - fundamental use case performed to access the platform. Other use cases include authentication as a previous step.
- View Knowledge Base - allows Customers to browse existing solutions and documentation, attempting to solve issues without creating a new ticket.
- Receive Automated Alert - the system or external platform generates an automatic alert, which can lead to ticket creation. This use case shows the integration with the eMagiz Platform.
- Submit Ticket - central use case for creating tickets. Both the Customer and the eMagiz Platform can initiate this scenario. It specializes in two more specific cases:
 - Submit RFC - create a ticket for a Request for Change.
 - Submit Incident - create a ticket to log an incident or problem.
- Create Internal Ticket - used by the Support Agent or Internal Consultant to register internal tasks that are not directly initiated by the Customer.

Assess Ticket - use case where the Support Agent or Internal Consultant analyses the ticket, determines its priority and category, and decides on the next steps.

Log Action Details - included use case that is invoked from ticket-handling scenarios. It is responsible for recording all actions taken on a ticket for audit and traceability purposes.

Accept Ticket and Deny Ticket - alternative outcomes after ticket assessment. In “Accept Ticket” the actor confirms that the ticket will be processed; in “Deny Ticket” the ticket is rejected, and the reason is recorded through “Log Action Details”.

Assign Ticket - assigns a ticket to a specific handler or team. This use case is typically executed after a ticket has been accepted and also includes logging the assignment change.

Manage Assigned Tickets - allows Support Agents and Internal Consultants to monitor and update tickets for which they are responsible, changing status, adding comments, and reassigning tickets.

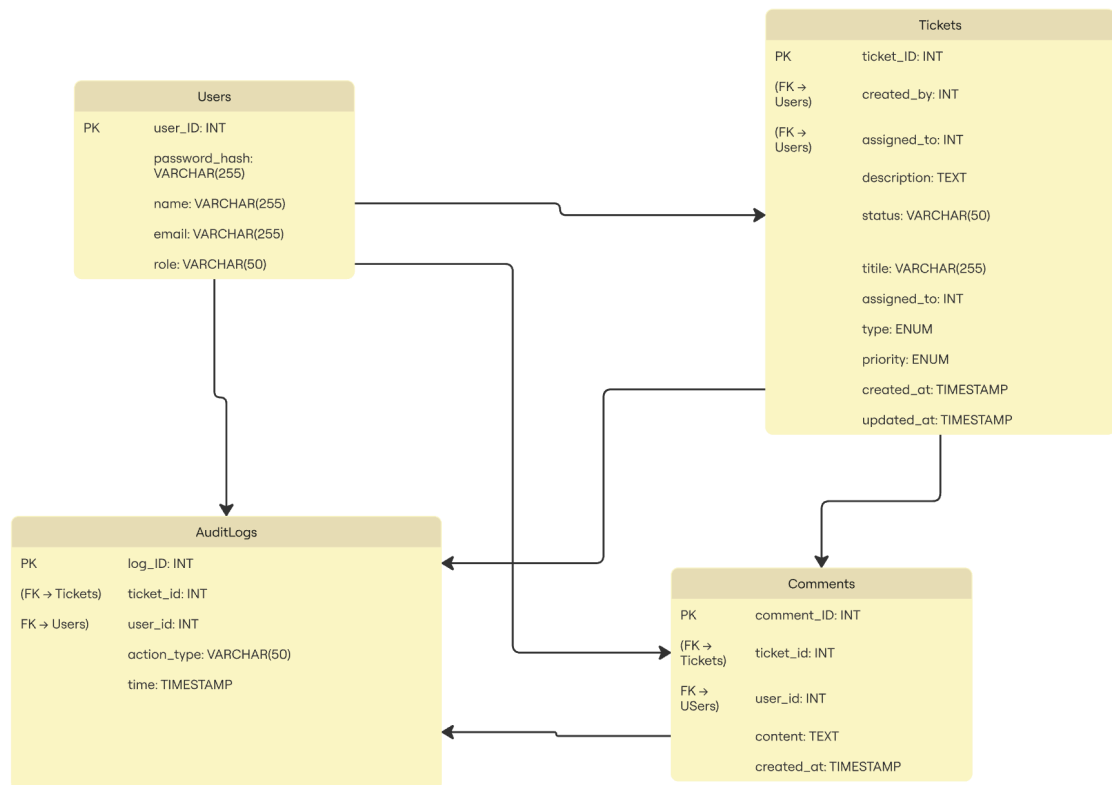
View Organization Tickets - gives the Internal Consultant an overview of all tickets for the organization, for example to analyse workload and service quality.

View Audit Log - enables the Internal Consultant to inspect the audit log that aggregates information from “Log Action Details” for all tickets. This is important for compliance, accountability, and incident analysis.

Relationships between use cases

- Authenticate User is modeled as an <<include>> for use cases that require a confirmed identity (such as Submit Ticket, Manage Assigned Tickets, View Organization Tickets). This represents a common preliminary step before most interactions with the system.
- Log Action Details is included (<<include>>) in use cases where changes to a ticket must be recorded (Assess Ticket, Accept Ticket, Deny Ticket, Assign Ticket, Manage Assigned Tickets). This highlights a reusable behaviour fragment.
- Accept Ticket and Deny Ticket logically extend (<<extend>>) the main “Assess Ticket” process: after assessment, one of these decisions is taken and additional steps (such as notifying the customer) are performed in the corresponding extension.

2. SQL Scheme



Description:

The relational schema for the eMagiz Ticketing System consists of four core tables: Users, Tickets, Comments, and AuditLogs. Together they support authentication, ticket lifecycle management, collaboration via comments, and full auditability of actions taken on tickets.

Users

The Users table models all authenticated actors of the system (customers, support agents, internal consultants):

- `user_id` : INT – primary key identifying each user.
- `password_hash` : VARCHAR(255) – hashed password used for authentication; the schema deliberately stores only hashes, not plain-text passwords.
- `name` : VARCHAR(255) – display name of the user.
- `email` : VARCHAR(255) – unique contact address used for notifications and login.
- `role` : VARCHAR(50) – application role of the user (for example, CUSTOMER, SUPPORT_AGENT, INTERNAL_CONSULTANT), aligning with the actors in the use case diagram.

This table is referenced by several foreign keys in other tables, since users can create tickets, be assigned to tickets, write comments, and perform actions that are logged.

Tickets

The Tickets table represents individual support tickets and change requests:

- `ticket_id` : INT – primary key of the ticket.
- `created_by` : INT – foreign key to `Users.user_id`, indicating who submitted the ticket (customer or internal user).
- `assigned_to` : INT – foreign key to `Users.user_id`, indicating the current assignee (support agent or internal consultant).
- `description` : TEXT – detailed description of the incident or request.
- `status` : VARCHAR(50) – current ticket status (for example, OPEN, IN_PROGRESS, RESOLVED, DENIED).
- `title` : VARCHAR(255) – short summary of the ticket.
- `type` : ENUM – ticket type, corresponding to use cases such as “Submit Incident” and “Submit RFC” (e.g., INCIDENT, RFC, INTERNAL).
- `priority` : ENUM – priority level (e.g., LOW, MEDIUM, HIGH, CRITICAL), used during the “Assess Ticket” use case.
- `created_at` : TIMESTAMP – creation timestamp.
- `updated_at` : TIMESTAMP – timestamp of the most recent update.

By referencing `Users` twice (`created_by` and `assigned_to`), the schema supports both the customer’s role as ticket originator and the support team’s role as handler, and the enumerated type and priority attributes reflect important domain concepts from the requirements.

Comments

The Comments table stores free-text discussion on tickets:

- `comment_id` : INT – primary key of the comment.

- ticket_id : INT – foreign key to Tickets.ticket_id, specifying which ticket the comment belongs to.
- user_id : INT – foreign key to Users.user_id, capturing who wrote the comment.
- content : TEXT – comment text.
- created_at : TIMESTAMP – time when the comment was added.

This table implements the many-to-one relationships “ticket has many comments” and “user can author many comments”. It supports collaboration between customers and support staff during the ticket’s lifecycle.

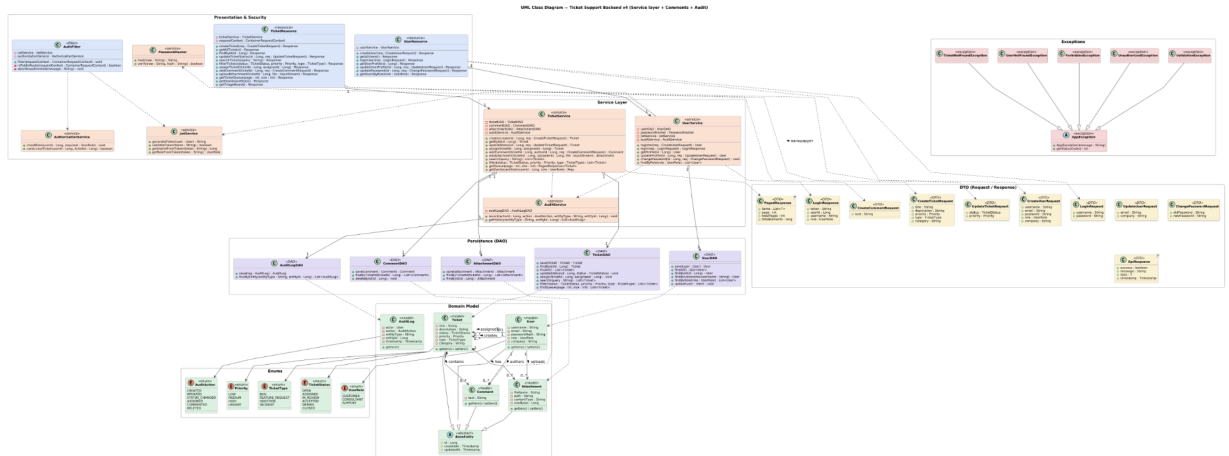
AuditLogs

The AuditLogs table provides a detailed history of actions performed on tickets, corresponding directly to the “Log Action Details” and “View Audit Log” use cases:

- log_id : INT – primary key of the log entry.
- ticket_id : INT – foreign key to Tickets.ticket_id, indicating which ticket the action relates to.
- user_id : INT – foreign key to Users.user_id, indicating who performed the action; this may be null for automated actions triggered by the platform, if required.
- action_type : VARCHAR(50) – short code or label describing the type of action (for example, CREATE, ACCEPT, DENY, ASSIGN, STATUS_CHANGE).
- time : TIMESTAMP – exact time the action occurred.

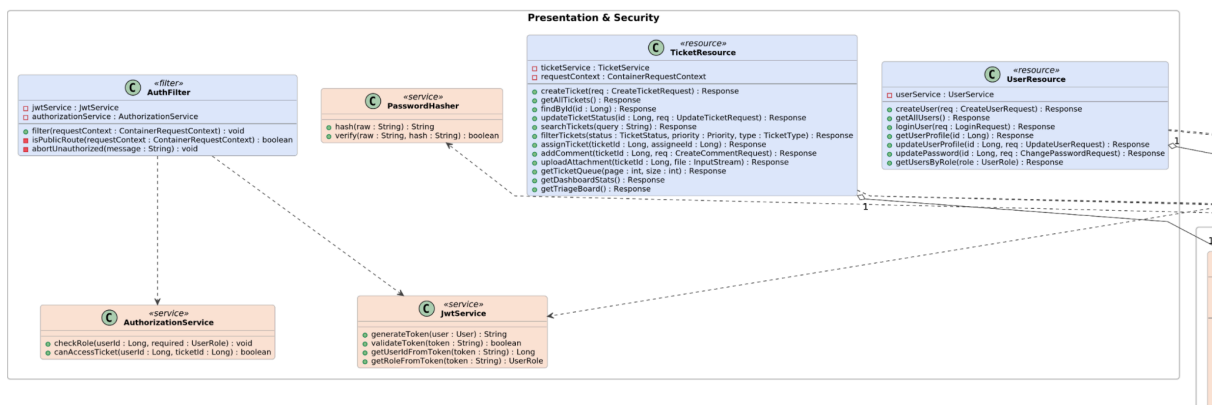
By separating audit entries into their own table, the schema supports normalization and scalability: arbitrary numbers of actions can be logged per ticket without duplicating ticket data, and the AuditLogs table can be efficiently queried to reconstruct the full history of a ticket or to generate organization-wide audit reports.

3. Class Diagram



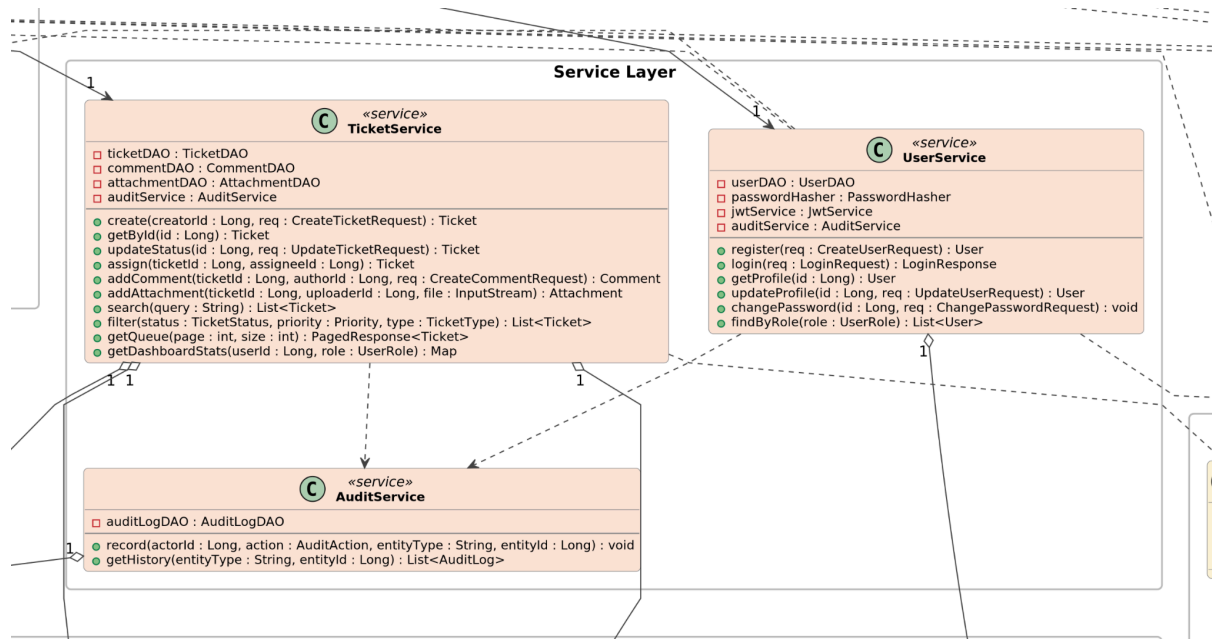
Description:

This picture shows the full structure of the Ticket Support Backend system. The system is divided into clear layers: resources and security, services, DAO/persistence, domain model, enums, DTOs and exceptions. The arrows show how the classes depend on each other. In simple words, requests come from the frontend to the resource classes, then go to services, then to DAO classes and finally to the database/domain objects.



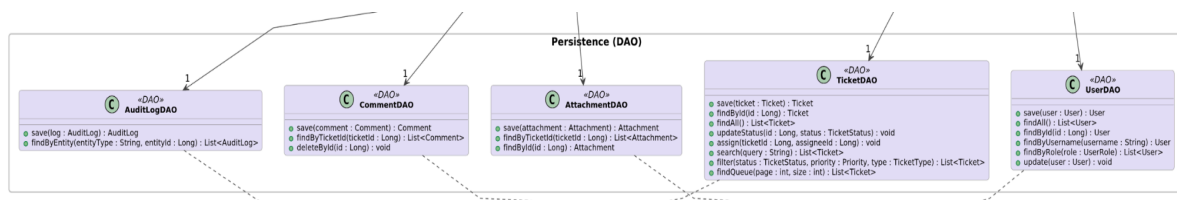
Description:

This layer contains the classes that receive HTTP requests and protect the application. `UserResource` handles user-related API requests such as registration, login, profile update and password change. `TicketResource` handles ticket-related API requests such as creating tickets, updating status, assigning tickets, adding comments and viewing ticket queues. `AuthFilter` checks incoming requests before they reach the API resources. It validates JWT tokens and blocks unauthorized users. `JwtService` creates and validates tokens and extracts user information from them. `AuthService` checks whether a user has permission to perform an action. `PasswordHasher` hashes and verifies passwords so that plain passwords are not stored.



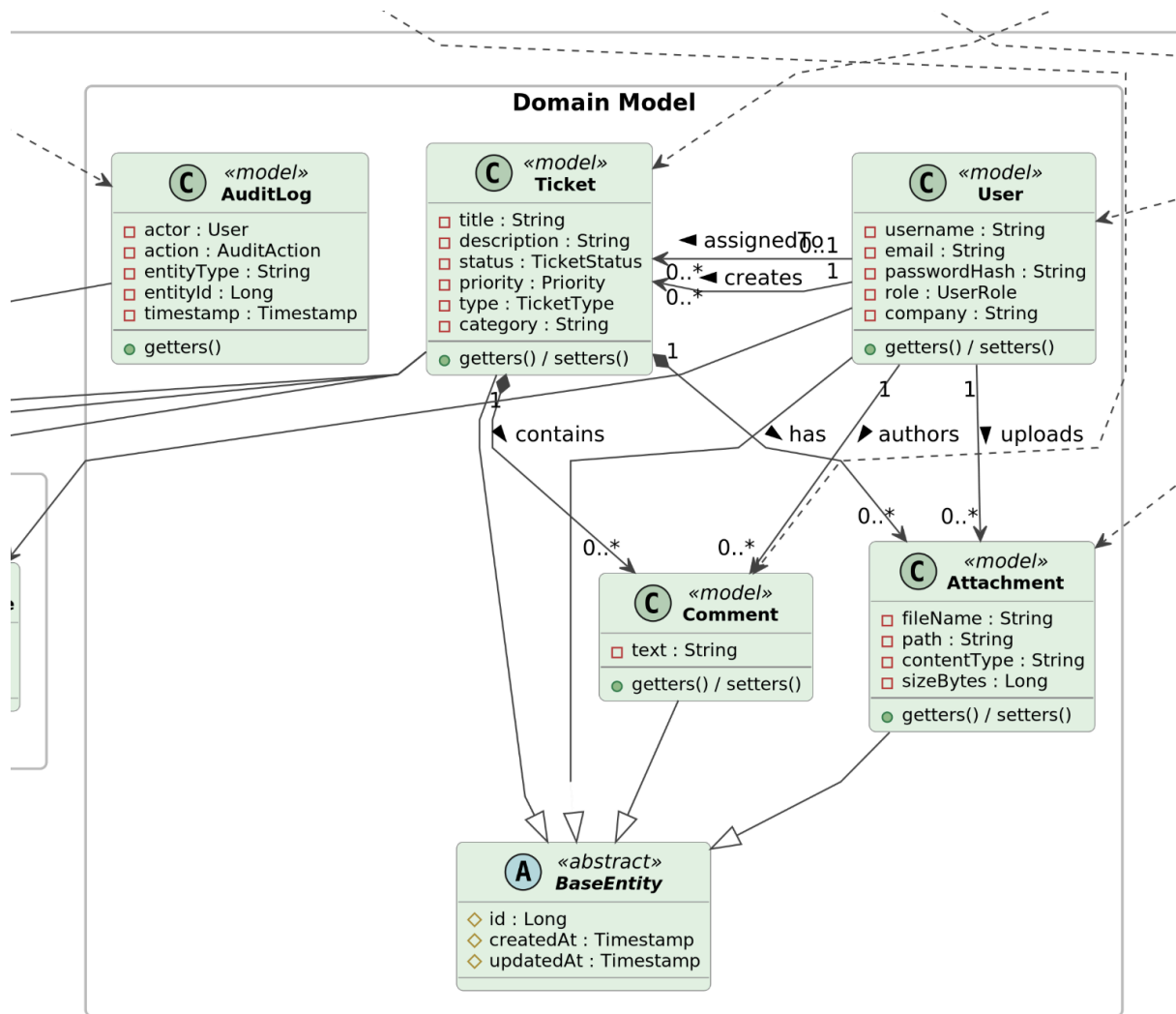
Description:

This layer contains the main business logic of the system. UserService controls user registration, login, profile updates, password changes and user searches. It uses UserDAO to access user data, PasswordHasher to handle passwords, JwtService to create login tokens and AuditService to record important actions. TicketService controls ticket operations such as creating tickets, updating status, assigning tickets, adding comments, uploading attachments, searching tickets and creating dashboard statistics. AuditService records actions such as ticket creation, status changes, assignments, comments and deletions. This makes the system more accountable and easier to track.



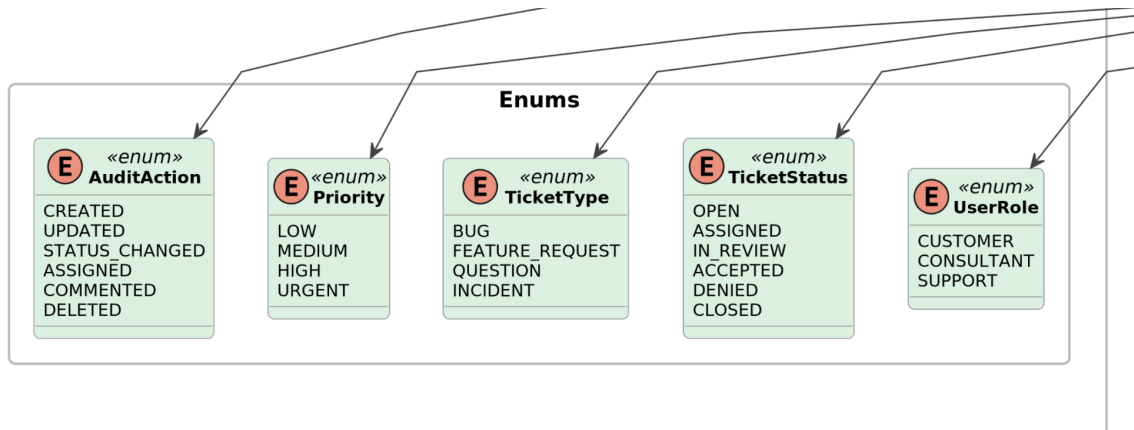
Description:

DAO means Data Access Object. These classes are responsible for communication with the database. UserDAO saves, finds and updates users. TicketDAO saves tickets, finds tickets by ID, searches tickets, filters tickets, updates status and assigns tickets. CommentDAO saves and retrieves comments for a ticket. AttachmentDAO saves and retrieves uploaded files. AuditLogDAO stores and retrieves audit log records. This layer separates database operations from business logic, which keeps the code cleaner and easier to maintain.



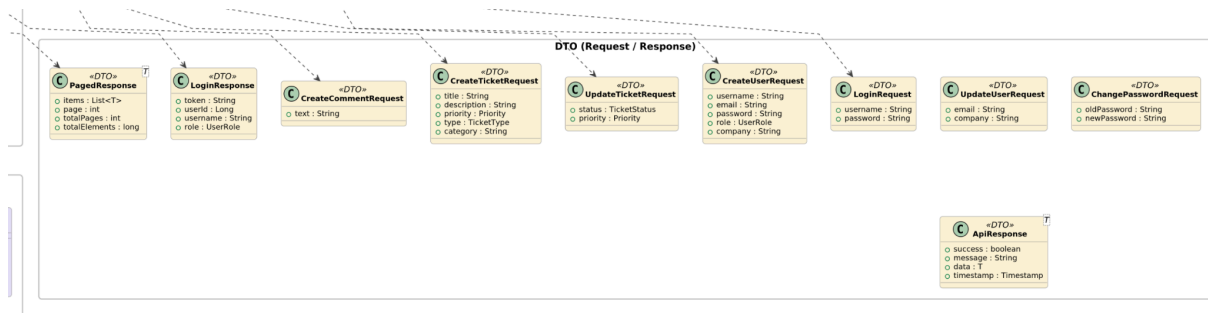
Description:

The domain model represents the main objects of the system. BaseEntity is the parent class that contains common fields such as id, createdAt and updatedAt. User represents a person using the system and stores username, email, passwordHash, role and company. Ticket represents a support request and contains title, description, status, priority, type and category. Comment represents a message written on a ticket. Attachment represents an uploaded file connected to a ticket. AuditLog represents a record of an important action, such as a user updating a ticket or changing its status.



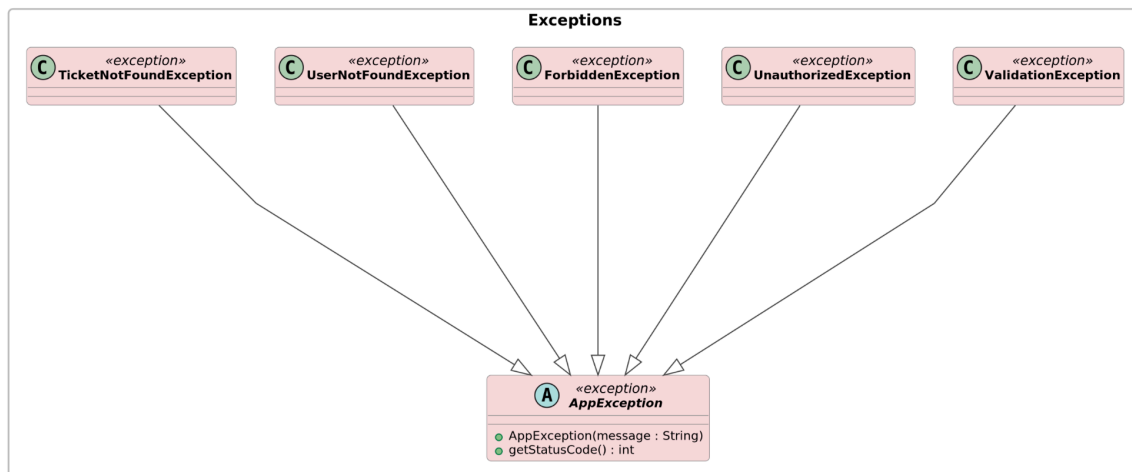
Description:

Enums define fixed allowed values in the system. UserRole defines roles such as CUSTOMER, CONSULTANT and SUPPORT. TicketStatus defines the possible states of a ticket, such as OPEN, ASSIGNED, IN_REVIEW, ACCEPTED, DENIED and CLOSED. Priority defines how urgent a ticket is: LOW, MEDIUM, HIGH, or URGENT. TicketType classifies the ticket as a BUG, FEATURE_REQUEST, QUESTION, or INCIDENT. AuditAction defines actions that can be recorded in the audit log, such as CREATED, UPDATED, STATUS_CHANGED, ASSIGNED, COMMENTED and DELETED.



Description:

DTO means Data Transfer Object. DTOs are used to send data between the frontend and backend without exposing the full domain model. CreateTicketRequest is used when a user creates a ticket. UpdateTicketRequest is used when a ticket status or priority is changed. CreateUserRequest is used for user registration, while UpdateUserRequest is used to update profile data. ChangePasswordRequest contains the old and new password. LoginRequest contains username and password. CreateCommentRequest contains the text of a new comment. LoginResponse returns the JWT token and user details after login. PagedResponse is used for paginated lists, and ApiResponse is a general wrapper containing success, message, data, and timestamp.



Description:

Exceptions represent error situations in the system. `AppException` is the base exception class. `ValidationException` is used when input data is invalid, for example when a required field is missing. `UnauthorizedException` is used when the user is not logged in or the token is invalid. `ForbiddenException` is used when the user is logged in but does not have permission to perform an action. `UserNotFoundException` is used when a requested user does not exist. `TicketNotFoundException` is used when a requested ticket does not exist. These exceptions help the backend return clear error responses to the frontend.